

C++

```
ComPtr<IDXGIAdapter> outputDxgiAdapter;  
DX::ThrowIfFailed(  
    dxgiFactory->EnumAdapters(0, &outputDxgiAdapter)  
);  
  
ComPtr<IDXGIOutput> dxgiOutput;  
DX::ThrowIfFailed(  
    outputDxgiAdapter->EnumOutputs(0, &dxgiOutput)  
);  
  
ComPtr<IDXGIOutput2> dxgiOutput2;  
DX::ThrowIfFailed(  
    dxgiOutput.As(&dxgiOutput2)  
);
```

The DXGI adapter supports overlays if the output adapter returns True for [SupportsOverlays](#).

C++

```
m_overlaySupportExists = dxgiOutput2->SupportsOverlays() ? true :  
false;
```

Note If the DXGI adapter supports overlays, continue to the next step. If the device does not support overlays, rendering with multiple swap chains will not be efficient. Instead, render the UI at reduced resolution in the same swap chain as real-time game content.

2. Create the foreground swap chain with

[IDXGIFactory2::CreateSwapChainForCoreWindow](#). The following options must be set in the [DXGI_SWAP_CHAIN_DESC1](#) supplied to the *pDesc* parameter:

- Specify the [DXGI_SWAP_CHAIN_FLAG_FOREGROUND_LAYER](#) swap chain flag to indicate a foreground swap chain.
- Use the [DXGI_ALPHA_MODE_PREMULTIPLIED](#) alpha mode flag. Foreground swap chains are always premultiplied.
- Set the [DXGI_SCALING_NONE](#) flag. Foreground swap chains always run at native resolution.

C++

```
foregroundSwapChainDesc.Flags = DXGI_SWAP_CHAIN_FLAG_FOREGROUND_LAYER;  
foregroundSwapChainDesc.Scaling = DXGI_SCALING_NONE;  
foregroundSwapChainDesc.AlphaMode = DXGI_ALPHA_MODE_PREMULTIPLIED; //  
Foreground swap chain alpha values must be premultiplied.
```

Note Set the [DXGI_SWAP_CHAIN_FLAG_FOREGROUND_LAYER](#) again every time the swap chain is resized.

C++

```
HRESULT hr = m_foregroundSwapChain->ResizeBuffers(  
    2, // Double-buffered swap chain.  
    static_cast<UINT>(m_d3dRenderTargetSize.Width),  
    static_cast<UINT>(m_d3dRenderTargetSize.Height),  
    DXGI_FORMAT_B8G8R8A8_UNORM,  
    DXGI_SWAP_CHAIN_FLAG_FOREGROUND_LAYER // The FOREGROUND_LAYER flag  
    cannot be removed with ResizeBuffers.  
);
```

- When two swap chains are being used, increase the maximum frame latency to 2 so that the DXGI adapter has time to present both swap chains simultaneously (within the same VSync interval).

C++

```
// Create a render target view of the foreground swap chain's back  
buffer.  
if (m_foregroundSwapChain)  
{  
    ComPtr<ID3D11Texture2D> foregroundBackBuffer;  
    DX::ThrowIfFailed(  
        m_foregroundSwapChain->GetBuffer(0,  
        IID_PPV_ARGS(&foregroundBackBuffer))  
    );  
  
    DX::ThrowIfFailed(  
        m_d3dDevice->CreateRenderTargetView(  
            foregroundBackBuffer.Get(),  
            nullptr,  
            &m_d3dForegroundRenderTargetView  
        )  
    );  
}
```

- Foreground swap chains always use premultiplied alpha. Each pixel's color values are expected to be already multiplied by the alpha value before the frame is

presented. For example, a 100% white BGRA pixel at 50% alpha is set to (0.5, 0.5, 0.5, 0.5).

The alpha premultiplication step can be done in the output-merger stage by applying an app blend state (see [ID3D11BlendState](#)) with the [D3D11_RENDER_TARGET_BLEND_DESC](#) structure's **SrcBlend** field set to **D3D11_SRC_ALPHA**. Assets with pre-multiplied alpha values can also be used.

If the alpha premultiplication step is not done, colors on the foreground swap chain will be brighter than expected.

5. Depending on whether the foreground swap chain was created, the Direct2D drawing surface for UI elements might need be associated with the foreground swap chain.

Creating render target views:

C++

```
// Create a render target view of the foreground swap chain's back
buffer.
if (m_foregroundSwapChain)
{
    ComPtr<ID3D11Texture2D> foregroundBackBuffer;
    DX::ThrowIfFailed(
        m_foregroundSwapChain->GetBuffer(0,
        IID_PPV_ARGS(&foregroundBackBuffer))
    );

    DX::ThrowIfFailed(
        m_d3dDevice->CreateRenderTargetView(
            foregroundBackBuffer.Get(),
            nullptr,
            &m_d3dForegroundRenderTargetView
        )
    );
}
```

Creating the Direct2D drawing surface:

C++

```
if (m_foregroundSwapChain)
{
    // Create a Direct2D target bitmap for the foreground swap chain.
    ComPtr<IDXGISurface2> dxgiForegroundSwapChainBackBuffer;
    DX::ThrowIfFailed(
        m_foregroundSwapChain->GetBuffer(0,
        IID_PPV_ARGS(&dxgiForegroundSwapChainBackBuffer))
    );
}
```

```

    );

    DX::ThrowIfFailed(
        m_d2dContext->CreateBitmapFromDxgiSurface(
            dxgiForegroundSwapChainBackBuffer.Get(),
            &bitmapProperties,
            &m_d2dTargetBitmap
        )
    );
}
else
{
    // Create a Direct2D target bitmap for the swap chain.
    ComPtr<IDXGISurface2> dxgiSwapChainBackBuffer;
    DX::ThrowIfFailed(
        m_swapChain->GetBuffer(0,
IID_PPV_ARGS(&dxgiSwapChainBackBuffer))
    );

    DX::ThrowIfFailed(
        m_d2dContext->CreateBitmapFromDxgiSurface(
            dxgiSwapChainBackBuffer.Get(),
            &bitmapProperties,
            &m_d2dTargetBitmap
        )
    );
}

m_d2dContext->SetTarget(m_d2dTargetBitmap.Get());

```

C++

```

// Create a render target view of the swap chain's back buffer.
ComPtr<ID3D11Texture2D> backBuffer;
DX::ThrowIfFailed(
    m_swapChain->GetBuffer(0, IID_PPV_ARGS(&backBuffer))
);

DX::ThrowIfFailed(
    m_d3dDevice->CreateRenderTargetView(
        backBuffer.Get(),
        nullptr,
        &m_d3dRenderTargetView
    )
);

```

6. Present the foreground swap chain together with the scaled swap chain used for real-time game content. Since frame latency was set to 2 for both swap chains, DXGI can present them both within the same VSync interval.

C++

```

// Present the contents of the swap chain to the screen.
void DX::DeviceResources::Present()
{
    // The first argument instructs DXGI to block until VSync, putting
    the application
    // to sleep until the next VSync. This ensures that we don't waste
    any cycles rendering
    // frames that will never be displayed to the screen.
    HRESULT hr = m_swapChain->Present(1, 0);

    if (SUCCEEDED(hr) && m_foregroundSwapChain)
    {
        m_foregroundSwapChain->Present(1, 0);
    }

    // Discard the contents of the render targets.
    // This is a valid operation only when the existing contents will
    be entirely
    // overwritten. If dirty or scroll rects are used, this call should
    be removed.
    m_d3dContext->DiscardView(m_d3dRenderTargetView.Get());
    if (m_foregroundSwapChain)
    {
        m_d3dContext-
>DiscardView(m_d3dForegroundRenderTargetView.Get());
    }

    // Discard the contents of the depth stencil.
    m_d3dContext->DiscardView(m_d3dDepthStencilView.Get());

    // If the device was removed either by a disconnection or a driver
    upgrade, we
    // must recreate all device resources.
    if (hr == DXGI_ERROR_DEVICE_REMOVED)
    {
        HandleDeviceLost();
    }
    else
    {
        DX::ThrowIfFailed(hr);
    }
}

```

Port from DirectX 9 to Universal Windows Platform (UWP)

Article • 10/20/2022

This section includes articles, overviews, and walkthroughs for porting DirectX 9 games to Universal Windows Platform (UWP).

Port your DirectX 9 game to UWP

- Reach the UWP audiences and monetize your game.
- Target a wide array of devices, all of which support at least the Direct3D 9.1 set of graphics features.
- Learn valuable new Windows game development skills - including Direct3D 11, the unified shader models, Windows APIs, XAudio2, touch input, C++/CX and more.

Where do I start?

- Visit [Moving from DirectX 9 to DirectX 11 and UWP](#) to learn what you should plan for in your game porting project, catch up on Direct3D 11 concepts, and understand how the features you're familiar with map to DirectX 11 UWP apps.
- Follow the [Port a simple Direct3D 9 app to DirectX 11 and UWP](#) walkthrough article for a direct comparison of Direct3D 9 and Direct3D 11 graphics frameworks. This walkthrough also has code for setting up an app window and viewport.
- See the [DirectX 11 porting FAQ](#) for answers to common questions about porting from DirectX 9.

Moving from DirectX 9 to DirectX 11 and Universal Windows Platform (UWP)

Article • 10/20/2022

This section has guidance on porting your DirectX 9 desktop game to DirectX 11 and Universal Windows Platform (UWP).

Topic	Description
Plan your DirectX port	Plan your game porting project from DirectX 9 to DirectX 11 and UWP: upgrade your graphics code, and put your game in the Windows Runtime environment.
Important changes from Direct3D 9 to Direct3D 11.1	This topic explains the high-level differences between DirectX 9 and DirectX 11.
Map DirectX 9 features to DirectX 11.1 APIs	Understand how the features your Direct3D 9 game uses will translate to Direct3D 11 and the UWP.

Plan your DirectX port

Article • 10/20/2022

Summary

- Plan your DirectX port
- [Important changes from Direct3D 9 to Direct3D 11](#)
- [Feature mapping](#)

Plan your game porting project from DirectX 9 to DirectX 11 and Universal Windows Platform (UWP): upgrade your graphics code, and put your game in the Windows Runtime environment.

Plan to port graphics code

Before you begin porting your game to UWP, it's important to ensure that your game does not have any holdovers from Direct3D 8. Ensure that your game doesn't have any remnants of the fixed function pipeline. For a complete list of deprecated features, including fixed pipeline functionality, see [Deprecated Features](#).

Upgrading from Direct3D 9 to Direct3D 11 is more than a search-and-replace change. You need to know the difference between the Direct3D device, device context, and graphics infrastructure, and learn about other important changes since Direct3D 9. You can start this process by reading the other topics in this section.

You must replace the D3DX and DXUT helper libraries with your own helper libraries, or with community tools. See the [Feature mapping](#) section for more info.

Note You can use the [DirectX Tool Kit](#) or [DirectXTex](#) to replace some functionality that was formerly provided by D3DX and DXUT.

Shaders written in assembly language should be upgraded to HLSL using shader model 4 level 9_1 or 9_3 functionality, and shaders written for the Effects library will need to be updated to a more recent version of HLSL syntax. See the [Feature mapping](#) section for more info.

Get familiar with the different [Direct3D feature levels](#). Feature levels classify a wide range of video hardware by defining sets of known functionality. Each set roughly

corresponds to versions of Direct3D, from 9.1 through 11.2. All feature levels use the DirectX 11 API.

Plan to port Win32 UI code to CoreWindow

UWP apps run in a window created for an app container, called a [CoreWindow](#). Your game controls the window by inheriting from [IFrameworkView](#), which requires less implementation details than a desktop window. Your game's main loop will be in the [IFrameworkView::Run](#) method.

The lifecycle of a UWP app is very different from a desktop app. You'll need to save the game often, because when a suspend event happens your app only has a limited amount of time to stop running code, and you want to make sure the player can get back to where they were right away when your app resumes. Games should save just often enough to maintain a continuous gameplay experience from resume, but not so often that the game saves impact framerate or cause the game to stutter. Your game will potentially need to load game state when the game resumes from a terminated state.

[DirectXMath](#) can be used as a replacement for D3DXMath and XNAMath, and it can come in handy if you need a math library. DirectXMath has fast, portable data types, and types that are aligned and packed for use with shaders.

Native libraries such as the [Interlocked API](#) have been expanded to support Arm intrinsics. If your game uses interlocked APIs, you can keep using them in DirectX 11 and UWP.

Our templates and code samples use new C++ features that you might not be familiar with yet. For example, asynchronous methods are used with [lambda expressions](#) to load Direct3D resources without blocking the UI thread.

There are two concepts you'll use often:

- Managed references ([^ operator](#)) and [managed classes](#) (ref classes) are a fundamental part of the Windows Runtime. You will need to use managed ref classes to interface with Windows Runtime components, for example [IFrameworkView](#) (more on that in the walkthrough).
- When working with Direct3D 11 COM interfaces, use the [Microsoft::WRL::ComPtr](#) template type to make COM pointers easier to use.

Important changes from Direct3D 9 to Direct3D 11

Article • 10/20/2022

Summary

- [Plan your DirectX port](#)
- Important changes from Direct3D 9 to Direct3D 11
- [Feature mapping](#)

This topic explains the high-level differences between DirectX 9 and DirectX 11.

Direct3D 11 is fundamentally the same type of API as Direct3D 9 - a low-level, virtualized interface into graphics hardware. It still allows you to perform graphics drawing operations on a variety of hardware implementations. The layout of the graphics API has changed since Direct3D 9; the concept of a device context has been expanded, and an API has been added specifically for graphics infrastructure. Resources stored on the Direct3D device have a novel mechanism for data polymorphism called a resource view.

Core API functions

In Direct3D 9 you had to create an interface to the Direct3D API before you could start using it. In your Direct3D 11 Universal Windows Platform (UWP) game, you call a static function called [D3D11CreateDevice](#) to create the device and the device context.

Devices and device context

A Direct3D 11 device represents a virtualized graphics adapter. It's used to create resources in video memory, for example: uploading textures to the GPU, creating views on texture resources and swap chains, and creating texture samplers. For a complete list of what a Direct3D 11 device interface is used for see [ID3D11Device](#) and [ID3D11Device1](#).

A Direct3D 11. device context is used to set pipeline state and generate rendering commands. For example, a Direct3D 11 rendering chain uses a device context to set up the rendering chain and draw the scene (see below). The device context is used to access (map) video memory used by Direct3D device resources; it's also used to update subresource data, for example constant buffer data. For a complete list of what a Direct3D 11 device context is used for see [ID3D11DeviceContext](#) and [ID3D11DeviceContext1](#). Note that most of our samples use an immediate context to

render directly to the device, but Direct3D 11 also supports deferred device contexts, which are primarily used for multithreading.

In Direct3D 11, the device handle and device context handle are both obtained by calling [D3D11CreateDevice](#). This method is also where you request a specific set of hardware features and retrieve information on Direct3D feature levels supported by the graphics adapter. See [Introduction to a Device in Direct3D 11](#) for more info on devices, device contexts, and threading considerations.

Device infrastructure, frame buffers, and render target views

In Direct3D 11, the device adapter and hardware configuration are set with the DirectX Graphics Infrastructure (DXGI) API using the [IDXGIAdapter](#) and [IDXGIDevice1](#) COM interfaces. Buffers and other window resources (visible or offscreen) are created and configured by specific DXGI interfaces; the [IDXGIFactory2](#) factory pattern implementation acquires DXGI resources such as the frame buffer. Since DXGI owns the swap chain, a DXGI interface is used to present frames to the screen - see [IDXGISwapChain1](#).

Use [IDXGIFactory2](#) to create a swap chain compatible with your game. You need to create a swap chain for a core window, or for composition (XAML interop), instead of creating a swap chain for an HWND.

Device resources and resource views

Direct3D 11 supports an additional level of polymorphism on video memory resources known as views. Essentially, where you had a single Direct3D 9 object for a texture, you now have two objects: the texture resource, which holds the data, and the resource view, which indicates how the view is used for rendering. A view based on a resource enables that resource to be used for a specific purpose. For example, a 2D texture resource is created as an [ID3D11Texture2D](#), then a shader resource view ([ID3D11ShaderResourceView](#)) is created on it so it can be used as a texture in a shader. A render target view ([ID3D11RenderTargetView](#)) can also be created on the same 2D texture resource so that it can be used as a drawing surface. In another example, the same pixel data is represented in 2 different pixel formats by using 2 separate views on a single texture resource.

The underlying resource must be created with properties that are compatible with the type of views that will be created from it. For example, if an [ID3D11RenderTargetView](#) is applied to a surface, that surface is created with the [D3D11_BIND_RENDER_TARGET](#) flag.

The surface also has to have a DXGI surface format compatible with rendering (see [DXGI_FORMAT](#)).

Most of the resources you use for rendering inherit from the [ID3D11Resource](#) interface, which inherits from [ID3D11DeviceChild](#). Vertex buffers, index buffers, constant buffers, and shaders are all Direct3D 11 resources. Input layouts and sampler states inherit directly from [ID3D11DeviceChild](#).

Resources views use a DXGI_FORMAT enum value to indicate the pixel format. Not every D3DFMT is supported as a DXGI_FORMAT. For example, there is no 24bpp RGB format in DXGI that is equivalent to D3DFMT_R8G8B8. There are also not BGR equivalents to every RGB format (DXGI_FORMAT_R10G10B10A2_UNORM is equivalent to D3DFMT_A2B10G10R10, but there's no direct equivalent to D3DFMT_A2R10G10B10). You should plan to convert any content in these legacy formats to supported formats at build-time. For a complete list of DXGI formats see the [DXGI_FORMAT](#) enumeration.

Direct3D device resources (and resource views) are created before the scene is rendered. Device contexts are used to set up the rendering chain, as explained below.

Device context and the rendering chain

In Direct3D 9 and Direct3D 10.x, there was a single Direct3D device object which managed resource creation, state, and drawing. In Direct3D 11, the Direct3D device interface still manages resource creation, but all state and drawing operations are handled by using a Direct3D device context. Here's an example of how the device context ([ID3D11DeviceContext1](#) interface) is used to set up the rendering chain:

- Set and clear render target views (and depth stencil view)
- Set the vertex buffer, index buffer, and input layout for the input assembler stage (IA stage)
- Bind vertex and pixel shaders to the pipeline
- Bind constant buffers to shaders
- Bind texture views and samplers to the pixel shader
- Draw the scene

When one of the [ID3D11DeviceContext::Draw](#) methods is called, the scene is drawn on the render target view. When you're done with all your drawing the DXGI adapter is used to present the completed frame by calling [IDXGISwapChain1::Present1](#).

State management

Direct3D 9 managed state settings with a large set of individual toggles set with the `SetRenderState`, `SetSamplerState`, and `SetTextureStageState` methods. Since Direct3D 11 does not support the legacy fixed-function pipeline, the `SetTextureStageState` is replaced by writing pixel shaders (PS). There is no equivalent to a Direct3D 9 state block. Direct3D 11 instead manages state through the use of 4 kinds of state objects which provide a more streamlined way to group the rendering state.

For example, instead of using `SetRenderState` with `D3DRS_ZENABLE`, you create a `DepthStencilState` object with this and other related state settings and use it to change state while rendering.

When porting Direct3D 9 applications to state objects, be aware that your various state combinations are represented as immutable state objects. They should be created once and re-used as long as they are valid.

Direct3D feature levels

Direct3D has a new mechanism for determining hardware support called feature levels. Feature levels simplify the task of figuring out what the graphics adapter can do by allowing you to request a well-defined set of GPU functionality. For example, the 9_1 feature level implements the functionality provided by Direct3D 9 graphics adapters, including shader model 2.x. Since 9_1 is the lowest feature level, you can expect all devices to support a vertex shader and a pixel shader, which were the same stages supported by the Direct3D 9 programmable shader model.

Your game will use [D3D11CreateDevice](#) to create the Direct3D device and device context. When you call this function you provide a list of feature levels that your game can support. It will return the highest supported feature level from that list. For example if your game can use BC4/BC5 textures (a feature of DirectX 10 hardware), you would include at least 9_1 and 10_0 in the list of supported feature levels. If the game is running on DirectX 9 hardware and BC4/BC5 textures can't be used, then [D3D11CreateDevice](#) will return 9_1. Then your game can fall back to a different texture format (and smaller textures).

If you decide to extend your Direct3D 9 game to support higher Direct3D feature levels then it's better to finish porting your existing Direct3D 9 graphics code first. After you have your game working in Direct3D 11, it's easier to add additional rendering paths with enhanced graphics.

See [Direct3D feature levels](#) for a detailed explanation of feature level support. See [Direct3D 11 Features](#) and [Direct3D 11.1 Features](#) for a full list of Direct3D 11 features.

Feature levels and the programmable pipeline

Hardware has continue to evolve since Direct3D 9, and several new optional stages have been added to the programmable graphics pipeline. The set of options you have for the graphics pipeline varies with the Direct3D feature level. Feature level 10.0 includes the geometry shader stage with optional stream out for multipass rendering on the GPU. Feature level 11_0 include the hull shader and domain shader for use with hardware tessellation. Feature level 11_0 also includes full support for DirectCompute shaders, while feature levels 10.x only include support for a limited form of DirectCompute.

All shaders are written in HLSL using a shader profile that corresponds to a Direct3D feature level. Shader profiles are upwards compatible, so an HLSL shader that compiles using `vs_4_0_level_9_1` or `ps_4_0_level_9_1` will work across all devices. Shader profiles are not downlevel compatible, so a shader compiled using `vs_4_1` will only work on feature level 10_1, 11_0, or 11_1 devices.

Direct3D 9 managed constants for shaders using a shared array with `SetVertexShaderConstant` and `SetPixelShaderConstant`. Direct3D 11 uses constant buffers, which are resources like a vertex buffer or index buffer. Constant buffers are designed to be updated efficiently. Instead of having all the shader constants organized into a single global array you organize your constants into logical groupings and manage them through one or more constant buffers. When you port your Direct3D 9 game to Direct3D 11, plan to organize your constant buffers so that you can update them appropriately. For example, group shader constants that aren't updated every frame into a separate constant buffer, so that you don't have to constantly upload that data to the graphics adapter along with your more dynamic shader constants.

Note Most Direct3D 9 applications made extensive use of shaders, but occasionally mixed in use of the legacy fixed-function behavior. Note that Direct3D 11 only uses a programmable shading model. The legacy fixed-function features of Direct3D 9 are deprecated.

Feedback

Was this page helpful?



Yes



No

[Provide product feedback](#) 

| [Get help at Microsoft Q&A](#)

Map DirectX 9 features to DirectX 11 APIs

Article • 10/20/2022

Understand how the features your Direct3D 9 game uses will translate to Direct3D 11 and the Universal Windows Platform (UWP).

Also see [Plan your DirectX port](#), and [Important changes from Direct3D 9 to Direct3D 11](#).

Mapping Direct3D 9 to DirectX 11 APIs

[Direct3D](#) is still the foundation of DirectX graphics, but the API has changed since DirectX 9:

- Microsoft DirectX Graphics Infrastructure (DXGI) is used to set up graphics adapters. Use [DXGI](#) to select buffer formats, create swap chains, present frames, and create shared resources. See [DXGI Overview](#).
- A Direct3D device context is used to set pipeline state and generate rendering commands. Most of our samples use an immediate context to render directly to the device; Direct3D 11 also supports multithreaded rendering, in which case deferred contexts are used. See [Introduction to a Device in Direct3D 11](#).
- Some features have been deprecated, most notably the fixed function pipeline. See [Deprecated Features](#).

For a full list of Direct3D 11 features, see [Direct3D 11 Features](#) and [Direct3D 11 Features](#).

Moving from Direct2D 9 to Direct2D 11

[Direct2D \(Windows\)](#) is still an important part of DirectX graphics and Windows. You can still use Direct2D to draw 2D games, and to draw overlays (HUDs) on top of Direct3D.

Direct2D runs on top of Direct3D; 2D games can be implemented using either API. For example, a 2D game implemented using Direct3D can use orthographic projection, set Z-values to control the drawing order of primitives, and use pixel shaders to add special effects.

Since Direct2D is based on Direct3D it also uses DXGI and device contexts. See [Direct2D API Overview](#).

The [DirectWrite](#) API adds support for formatted text using Direct2D. See [Introducing DirectWrite](#).

Replace deprecated helper libraries

D3DX and DXUT are deprecated and cannot be used by UWP games. These helper libraries provided resources for tasks such as texture loading and mesh loading.

- The [Simple port from Direct3D 9 to UWP](#) walkthrough demonstrates how to set up a window, initialize Direct3D, and do basic 3D rendering.
- The [Simple UWP game with DirectX](#) walkthrough demonstrates common game programming tasks including graphics, loading files, UI, controls, and sound.
- The [DirectX Tool Kit](#) [↗](#) community project offers helper classes for use with Direct3D 11 and UWP apps.

Move shader programs from FX to HLSL

The D3DX utility library (D3DX 9, D3DX 10, and D3DX 11), including Effects, is deprecated for UWP. All DirectX games for UWP drive the graphics pipeline using [HLSL](#) without Effects.

Visual Studio still uses FXC under the hood to compile shader objects. UWP game shaders are compiled ahead of time. The bytecode is loaded at runtime, then each shader resource is bound to the graphics pipeline during the appropriate rendering pass. Shaders should be moved to their own separate .HLSL files and rendering techniques should be implemented in your C++ code.

For a quick look at loading shader resources see [Simple port from Direct3D 9 to UWP](#).

Direct3D 11 introduced Shader Model 5, which requires Direct3D feature level 11_0 (or above). See [HLSL Shader Model 5 Features for Direct3D 11](#).

Replace XNAMath and D3DXMath

Code using XNAMath (or D3DXMath) should be migrated to [DirectXMath](#). DirectXMath includes types that are portable across x86, x64, and Arm. See [Code Migration from the XNA Math Library](#).

Note that DirectXMath float types are convenient for use with shaders. For example [XMFLOAT4](#) and [XMFLOAT4X4](#) conveniently align data for constant buffers.

Replace DirectSound with XAudio2 (and background audio)

DirectSound is not supported for UWP:

- Use [XAudio2](#) to add sound effects to your game.

Replace DirectInput with XInput and Windows Runtime APIs

DirectInput is not supported for UWP:

- Use [CoreWindow](#) input event callbacks for mouse, keyboard, and touch input.
- Use [XInput](#) 1.4 for game controller support (and game controller headset support). If you are using a shared code base for desktop and UWP, see [XInput Versions](#) for information on backwards compatibility.
- Register for [EdgeGesture](#) events if your game needs to use the app bar.

Use Microsoft Media Foundation instead of DirectShow

DirectShow is no longer part of the DirectX API (or the Windows API). [Microsoft Media Foundation](#) provides video content to Direct3D using shared surfaces. See [Direct3D 11 Video APIs](#).

Replace DirectPlay with networking code

Microsoft DirectPlay has been deprecated. If your game uses network services, you need to provide networking code that complies with UWP requirements. Use the following APIs:

- [Win32 and COM for UWP apps \(networking\) \(Windows\)](#)
- [Windows.Networking namespace \(Windows\)](#)
- [Windows.Networking.Sockets namespace \(Windows\)](#)
- [Windows.Networking.Connectivity namespace \(Windows\)](#)
- [Windows.ApplicationModel.Background namespace \(Windows\)](#)

The following articles help you add networking features and declare support for networking in your app's package manifest.

- [Connecting with sockets \(UWP apps using C#/VB/C++ and XAML\) \(Windows\)](#)
- [Connecting with WebSockets \(UWP apps using C#/VB/C++ and XAML\) \(Windows\)](#)
- [Connecting to web services \(UWP apps using C#/VB/C++ and XAML\) \(Windows\)](#)
- [Networking basics](#)

Note that all UWP apps (including games) use specific types of background tasks to maintain connectivity while the app is suspended. If your game needs to maintain connection state while suspended see [Networking basics](#).

Function mapping

Use the following table to help convert code from Direct3D 9 to Direct3D 11. This can also help distinguish between the device and device context.

Direct3D9	Direct3D 11 Equivalent
IDirect3DDevice9	ID3D11Device2 ID3D11DeviceContext2 The graphics pipeline stages are described in Graphics Pipeline .
IDirect3D9	IDXGIFactory2 IDXGIAdapter2 IDXGIDevice3
IDirect3DDevice9::Present	IDXGISwapChain1::Present1
IDirect3DDevice9::TestCooperativeLevel	Call IDXGISwapChain1::Present1 with the <code>DXGI_PRESENT_TEST</code> flag set.
IDirect3DBaseTexture9	ID3D11Buffer
IDirect3DTexture9	ID3D11Texture1D
IDirect3DCubeTexture9	ID3D11Texture2D
IDirect3DVolumeTexture9	ID3D11Texture3D
IDirect3DIndexBuffer9	ID3D11ShaderResourceView
IDirect3DVertexBuffer9	ID3D11RenderTargetView ID3D11DepthStencilView

Direct3D9	Direct3D 11 Equivalent
IDirect3DVertexShader9	ID3D11VertexShader
IDirect3DPixelShader9	ID3D11PixelShader
IDirect3DVertexDeclaration9	ID3D11InputLayout
IDirect3DDevice9::SetRenderState	ID3D11BlendState1
IDirect3DDevice9::SetSamplerState	ID3D11DepthStencilState
	ID3D11RasterizerState1
	ID3D11SamplerState
IDirect3DDevice9::DrawIndexedPrimitive	ID3D11DeviceContext::Draw
IDirect3DDevice9::DrawPrimitive	ID3D11DeviceContext::DrawIndexed
	ID3D11DeviceContext::DrawIndexedInstanced
	ID3D11DeviceContext::DrawInstanced
	ID3D11DeviceContext::IASetPrimitiveTopology
	ID3D11DeviceContext::DrawAuto
IDirect3DDevice9::BeginScene	No direct equivalent
IDirect3DDevice9::EndScene	
IDirect3DDevice9::DrawPrimitiveUP	
IDirect3DDevice9::DrawIndexedPrimitiveUP	
IDirect3DDevice9::ShowCursor	Use standard cursor APIs.
IDirect3DDevice9::SetCursorPosition	
IDirect3DDevice9::SetCursorProperties	
IDirect3DDevice9::Reset	LOST device and POOL_MANAGED no longer exist. IDXGISwapChain1::Present1 can fail with a DXGI_ERROR_DEVICE_REMOVED return value.

Direct3D9	Direct3D 11 Equivalent
IDirect3DDevice9:DrawRectPatch IDirect3DDevice9:DrawTriPatch IDirect3DDevice9:LightEnable IDirect3DDevice9:MultiplyTransform IDirect3DDevice9:SetLight IDirect3DDevice9:SetMaterial IDirect3DDevice9:SetNPatchMode IDirect3DDevice9:SetTransform IDirect3DDevice9:SetFVF IDirect3DDevice9:SetTextureStageState	The fixed-function pipeline has been deprecated.
IDirect3DDevice9:CheckDepthStencilMatch IDirect3DDevice9:CheckDeviceFormat IDirect3DDevice9:GetDeviceCaps IDirect3DDevice9:ValidateDevice	Capability bits are replaced by feature levels. Only a few format and feature usage cases are optional for any given feature level. These can be checked with ID3D11Device::CheckFeatureSupport and ID3D11Device::CheckFormatSupport.

Surface format mapping

Use the following table to convert Direct3D 9 formats into DXGI formats.

Direct3D 9 Format	Direct3D 11 Format
D3DFMT_UNKNOWN	DXGI_FORMAT_UNKNOWN
D3DFMT_R8G8B8	Not available
D3DFMT_A8R8G8B8	DXGI_FORMAT_B8G8R8A8_UNORM DXGI_FORMAT_B8G8R8A8_UNORM_SRGB
D3DFMT_X8R8G8B8	DXGI_FORMAT_B8G8R8X8_UNORM DXGI_FORMAT_B8G8R8X8_UNORM_SRGB
D3DFMT_R5G6B5	DXGI_FORMAT_B5G6R5_UNORM
D3DFMT_X1R5G5B5	Not available

Direct3D 9 Format	Direct3D 11 Format
D3DFMT_A1R5G5B5	DXGI_FORMAT_B5G5R5A1_UNORM
D3DFMT_A4R4G4B4	DXGI_FORMAT_B4G4R4A4_UNORM
D3DFMT_R3G3B2	Not available
D3DFMT_A8	DXGI_FORMAT_A8_UNORM
D3DFMT_A8R3G3B2	Not available
D3DFMT_X4R4G4B4	Not available
D3DFMT_A2B10G10R10	DXGI_FORMAT_R10G10B10A2
D3DFMT_A8B8G8R8	DXGI_FORMAT_R8G8B8A8_UNORM DXGI_FORMAT_R8G8B8A8_UNORM_SRGB
D3DFMT_X8B8G8R8	Not available
D3DFMT_G16R16	DXGI_FORMAT_R16G16_UNORM
D3DFMT_A2R10G10B10	Not available
D3DFMT_A16B16G16R16	DXGI_FORMAT_R16G16B16A16_UNORM
D3DFMT_A8P8	Not available
D3DFMT_P8	Not available
D3DFMT_L8	DXGI_FORMAT_R8_UNORM Note Use .r swizzle in shader to duplicate red to other components to get Direct3D 9 behavior.
D3DFMT_A8L8	DXGI_FORMAT_R8G8_UNORM Note Use swizzle .rrrg in shader to duplicate red and move green to the alpha components to get Direct3D 9 behavior.
D3DFMT_A4L4	Not available

Direct3D 9 Format	Direct3D 11 Format
D3DFMT_V8U8	DXGI_FORMAT_R8G8_SNORM
D3DFMT_L6V5U5	Not available
D3DFMT_X8L8V8U8	Not available
D3DFMT_Q8W8V8U8	DXGI_FORMAT_R8G8B8A8_SNORM
D3DFMT_V16U16	DXGI_FORMAT_R16G16_SNORM
D3DFMT_W11V11U10	Not available
D3DFMT_A2W10V10U10	Not available
D3DFMT_UYVY	Not available
D3DFMT_R8G8_B8G8	DXGI_FORMAT_G8R8_G8B8_UNORM Note In Direct3D 9 the data was scaled up by 255.0f, but this can be handled in the shader.
D3DFMT_YUY2	Not available
D3DFMT_G8R8_G8B8	DXGI_FORMAT_R8G8_B8G8_UNORM Note In Direct3D 9 the data was scaled up by 255.0f, but this can be handled in the shader.
D3DFMT_DXT1	DXGI_FORMAT_BC1_UNORM & DXGI_FORMAT_BC1_UNORM_SRGB

Direct3D 9 Format	Direct3D 11 Format
D3DFMT_DXT2	DXGI_FORMAT_BC1_UNORM & DXGI_FORMAT_BC1_UNORM_SRGB Note DXT1 and DXT2 are the same from an API/hardware perspective. The only difference is whether premultiplied alpha is used, which can be tracked by an application and doesn't need a separate format.
D3DFMT_DXT3	DXGI_FORMAT_BC2_UNORM & DXGI_FORMAT_BC2_UNORM_SRGB
D3DFMT_DXT4	DXGI_FORMAT_BC2_UNORM & DXGI_FORMAT_BC2_UNORM_SRGB Note DXT3 and DXT4 are the same from an API/hardware perspective. The only difference is whether premultiplied alpha is used, which can be tracked by an application and doesn't need a separate format.
D3DFMT_DXT5	DXGI_FORMAT_BC3_UNORM & DXGI_FORMAT_BC3_UNORM_SRGB
D3DFMT_D16 & D3DFMT_D16_LOCKABLE	DXGI_FORMAT_D16_UNORM
D3DFMT_D32	Not available
D3DFMT_D15S1	Not available
D3DFMT_D24S8	Not available
D3DFMT_D24X8	Not available
D3DFMT_D24X4S4	Not available
D3DFMT_D16	DXGI_FORMAT_D16_UNORM
D3DFMT_D32F_LOCKABLE	DXGI_FORMAT_D32_FLOAT
D3DFMT_D24FS8	Not available

Direct3D 9 Format	Direct3D 11 Format
D3DFMT_S1D15	Not available
D3DFMT_S8D24	DXGI_FORMAT_D24_UNORM_S8_UINT
D3DFMT_X8D24	Not available
D3DFMT_X4S4D24	Not available
D3DFMT_L16	DXGI_FORMAT_R16_UNORM Note Use .r swizzle in shader to duplicate red to other components to get D3D9 behavior.
D3DFMT_INDEX16	DXGI_FORMAT_R16_UINT
D3DFMT_INDEX32	DXGI_FORMAT_R32_UINT
D3DFMT_Q16W16V16U16	DXGI_FORMAT_R16G16B16A16_SNORM
D3DFMT_MULTI2_ARGB8	Not available
D3DFMT_R16F	DXGI_FORMAT_R16_FLOAT
D3DFMT_G16R16F	DXGI_FORMAT_R16G16_FLOAT
D3DFMT_A16B16G16R16F	DXGI_FORMAT_R16G16B16A16_FLOAT
D3DFMT_R32F	DXGI_FORMAT_R32_FLOAT
D3DFMT_G32R32F	DXGI_FORMAT_R32G32_FLOAT
D3DFMT_A32B32G32R32F	DXGI_FORMAT_R32G32B32A32_FLOAT
D3DFMT_CxV8U8	Not available
D3DDECLTYPE_FLOAT1	DXGI_FORMAT_R32_FLOAT
D3DDECLTYPE_FLOAT2	DXGI_FORMAT_R32G32_FLOAT
D3DDECLTYPE_FLOAT3	DXGI_FORMAT_R32G32B32_FLOAT
D3DDECLTYPE_FLOAT4	DXGI_FORMAT_R32G32B32A32_FLOAT
D3DDECLTYPED3DCOLOR	Not available

Direct3D 9 Format	Direct3D 11 Format
D3DDECLTYPE_UBYTE4	DXGI_FORMAT_R8G8B8A8_UINT Note The shader gets UINT values, but if Direct3D 9 style integral floats are needed (0.0f, 1.0f... 255.f), UINT can just be converted to float32 in the shader.
D3DDECLTYPE_SHORT2	DXGI_FORMAT_R16G16_SINT Note The shader gets SINT values, but if Direct3D 9 style integral floats are needed, SINT can just be converted to float32 in the shader.
D3DDECLTYPE_SHORT4	DXGI_FORMAT_R16G16B16A16_SINT Note The shader gets SINT values, but if Direct3D 9 style integral floats are needed, SINT can just be converted to float32 in the shader.
D3DDECLTYPE_UBYTE4N	DXGI_FORMAT_R8G8B8A8_UNORM
D3DDECLTYPE_SHORT2N	DXGI_FORMAT_R16G16_SNORM
D3DDECLTYPE_SHORT4N	DXGI_FORMAT_R16G16B16A16_SNORM
D3DDECLTYPE_USHORT2N	DXGI_FORMAT_R16G16_UNORM
D3DDECLTYPE_USHORT4N	DXGI_FORMAT_R16G16B16A16_UNORM
D3DDECLTYPE_UDEC3	Not available
D3DDECLTYPE_DEC3N	Not available
D3DDECLTYPE_FLOAT16_2	DXGI_FORMAT_R16G16_FLOAT
D3DDECLTYPE_FLOAT16_4	DXGI_FORMAT_R16G16B16A16_FLOAT

Direct3D 9 Format	Direct3D 11 Format
FourCC 'ATI1'	DXGI_FORMAT_BC4_UNORM Note Requires Feature Level 10.0 or later
FourCC 'ATI2'	DXGI_FORMAT_BC5_UNORM Note Requires Feature Level 10.0 or later

Additional mapping info

- `IDirect3DDevice9::SetCursorPosition` is replaced by [SetCursorPos](#).
- `IDirect3DDevice9::SetCursorProperties` is replaced by [SetCursor](#).
- `IDirect3DDevice9::SetIndices` is replaced by [ID3D11DeviceContext::IASetIndexBuffer](#).
- `IDirect3DDevice9::SetRenderTarget` is replaced by [ID3D11DeviceContext::OMSetRenderTargets](#).
- `IDirect3DDevice9::SetScissorRect` is replaced by [ID3D11DeviceContext::RSSetScissorRects](#).
- `IDirect3DDevice9::SetStreamSource` is replaced by [ID3D11DeviceContext::IASetVertexBuffers](#).
- `IDirect3DDevice9::SetVertexDeclaration` is replaced by [ID3D11DeviceContext::IASetInputLayout](#).
- `IDirect3DDevice9::SetViewport` is replaced by [ID3D11DeviceContext::RSSetViewports](#).
- `IDirect3DDevice9::ShowCursor` is replaced by [ShowCursor](#).

Control of the video card's hardware gamma ramp through `IDirect3DDevice9::SetGammaRamp` is replaced by `IDXGIOutput::SetGammaControl`. See [Using gamma correction](#).

`IDirect3DDevice9::ProcessVertices` is replaced by the Stream-Output functionality of Geometry Shaders. See [Getting started with the Stream-Output Stage](#).

The method **IDirect3DDevice9::SetClipPlane** to set user clip-planes was replaced by either the HLSL **SV_ClipDistance** vertex shader output semantic (see [Semantics](#)), available in VS_4_0 and up, or the new HLSL clipplanes function attribute (see [User clip planes on feature level 9 hardware](#)).

IDirect3DDevice9::SetPaletteEntries and **IDirect3DDevice9::SetCurrentTexturePalette** are deprecated. Replace these with a pixel shader that looks up colors in a 256x1 **R8G8B8A8** texture instead.

Fixed-function tessellation functions like **DrawRectPatch**, **DrawTriPatch**, **SetNPatchMode**, and **DeletePatch** are deprecated. Replace these with programmable-pipeline SM5.0 Tessellation shaders (if hardware supports tessellation shaders).

IDirect3DDevice9::SetFVF, and FVF codes, are no longer supported. You should port from D3D8/D3D9 FVF codes to D3D9 Vertex Declarations before porting to D3D11 Input Layouts.

All of the **D3DDECLTYPE** types that are not directly supported can be emulated fairly efficiently with a small number of bitwise operations at the beginning of a vertex shader in VS_4_0 and up.

Walkthrough: Port a simple Direct3D 9 app to DirectX 11 and Universal Windows Platform (UWP)

Article • 10/20/2022

This porting exercise shows how to bring a simple rendering framework from Direct3D 9 to Direct3D 11 and Universal Windows Platform (UWP).

Topic	Description
Initialize Direct3D 11	Shows how to convert Direct3D 9 initialization code to Direct3D 11, including how to get handles to the Direct3D device and the device context and how to use DXGI to set up a swap chain.
Convert the rendering framework	Shows how to convert a simple rendering framework from Direct3D 9 to Direct3D 11, including how to port geometry buffers, how to compile and load HLSL shader programs, and how to implement the rendering chain in Direct3D 11.
Port the game loop	Shows how to implement a window for a UWP game and how to bring over the game loop, including how to build an IFrameworkView to control a full-screen CoreWindow .

This topic walks through two code paths that perform the same basic graphics task: display a rotating vertex-shaded cube. In both cases, the code covers the following process:

1. Creating a Direct3D device and a swap chain.
2. Creating a vertex buffer, and an index buffer, to represent a colorful cube mesh.
3. Creating a vertex shader that transforms vertices to screen space, a pixel shader that blends color values, compiling the shaders, and loading the shaders as Direct3D resources.
4. Implementing the rendering chain and presenting the drawn cube to the screen.
5. Creating a window, starting a main loop, and taking care of window message processing.

Upon completing this walkthrough, you should be familiar with the following basic differences between Direct3D 9 and Direct3D 11:

- The separation of device, device context, and graphics infrastructure.
- The process of compiling shaders, and loading shader bytecode at runtime.
- How to configure per-vertex data for the Input Assembler (IA) stage.
- How to use an [IFrameworkView](#) to create a CoreWindow view.

Note that this walkthrough uses [CoreWindow](#) for simplicity, and does not cover XAML interop.

Prerequisites

You should [Prepare your dev environment for UWP DirectX game development](#). You don't need a template yet, but you'll need Microsoft Visual Studio 2015 to load the code samples for this walkthrough.

Visit [Porting concepts and considerations](#) to gain a better understanding of the DirectX 11 and UWP programming concepts shown in this walkthrough.

Related topics

Direct3D

- [Writing HLSL Shaders in Direct3D 9](#)
- [DirectX game project templates](#)

Microsoft Store

- [Microsoft::WRL::ComPtr](#)
- [Handle to Object Operator \(^\)](#)

Initialize Direct3D 11

Article • 10/20/2022

Summary

- Part 1: Initialize Direct3D 11
- [Part 2: Convert the rendering framework](#)
- [Part 3: Port the game loop](#)

Shows how to convert Direct3D 9 initialization code to Direct3D 11, including how to get handles to the Direct3D device and the device context and how to use DXGI to set up a swap chain. Part 1 of the [Port a simple Direct3D 9 app to DirectX 11 and Universal Windows Platform \(UWP\)](#) walkthrough.

Initialize the Direct3D device

In Direct3D 9, we created a handle to the Direct3D device by calling [IDirect3D9::CreateDevice](#). We started by getting a pointer to [IDirect3D9 interface](#) and we specified a number of parameters to control the configuration of the Direct3D device and the swap chain. Before doing this we called [GetDeviceCaps](#) to make sure we weren't asking the device to do something it couldn't do.

Direct3D 9

C++

```
UINT32 AdapterOrdinal = 0;
D3DDEVTYPE DeviceType = D3DDEVTYPE_HAL;
D3DCAPS9 caps;
m_pD3D->GetDeviceCaps(AdapterOrdinal, DeviceType, &caps); // caps bits

D3DPRESENT_PARAMETERS params;
ZeroMemory(&params, sizeof(D3DPRESENT_PARAMETERS));

// Swap chain parameters:
params.hDeviceWindow = m_hWnd;
params.AutoDepthStencilFormat = D3DFMT_D24X8;
params.BackBufferFormat = D3DFMT_X8R8G8B8;
params.MultiSampleQuality = D3DMULTISAMPLE_NONE;
params.MultiSampleType = D3DMULTISAMPLE_NONE;
params.SwapEffect = D3DSWAPEFFECT_DISCARD;
params.Windowed = true;
params.PresentationInterval = 0;
params.BackBufferCount = 2;
params.BackBufferWidth = 0;
params.BackBufferHeight = 0;
```

```

params.EnableAutoDepthStencil = true;
params.Flags = 2;

m_pD3D->CreateDevice(
    0,
    D3DDEVTYPE_HAL,
    m_hWnd,
    64,
    &params,
    &m_pd3dDevice
);

```

In Direct3D 11, the device context and graphics infrastructure is considered separate from the device itself. Initialization is divided into multiple steps.

First we create the device. We get a list of the feature levels the device supports - this informs most of what we need to know about the GPU. Also, we don't need to create an interface just to access Direct3D. Instead we use the [D3D11CreateDevice](#) core API. This gives us a handle to the device and the device's immediate context. The device context is used to set pipeline state and generate rendering commands.

After creating the Direct3D 11 device and context, we can take advantage of COM pointer functionality to get the most recent version of the interfaces, which include additional capability and are always recommended.

Note D3D_FEATURE_LEVEL_9_1 (which corresponds to shader model 2.0) is the minimum level your Microsoft Store game is required to support. (Your game's Arm packages will fail certification if you don't support 9_1.) If your game also includes a rendering path for shader model 3 features, then you should include D3D_FEATURE_LEVEL_9_3 in the array.

Direct3D 11

C++

```

// This flag adds support for surfaces with a different color channel
// ordering than the API default. It is required for compatibility with
// Direct2D.
UINT creationFlags = D3D11_CREATE_DEVICE_BGRA_SUPPORT;

#ifdef _DEBUG
// If the project is in a debug build, enable debugging via SDK Layers.
creationFlags |= D3D11_CREATE_DEVICE_DEBUG;
#endif

```



```

// This example only uses feature level 9.1.
D3D_FEATURE_LEVEL featureLevels[] =
{
    D3D_FEATURE_LEVEL_9_1
};

// Create the Direct3D 11 API device object and a corresponding context.
ComPtr<ID3D11Device> device;
ComPtr<ID3D11DeviceContext> context;
D3D11CreateDevice(
    nullptr, // Specify nullptr to use the default adapter.
    D3D_DRIVER_TYPE_HARDWARE,
    nullptr,
    creationFlags,
    featureLevels,
    ARRAYSIZE(featureLevels),
    D3D11_SDK_VERSION, // UWP apps must set this to D3D11_SDK_VERSION.
    &device, // Returns the Direct3D device created.
    nullptr,
    &context // Returns the device immediate context.
);

// Store pointers to the Direct3D 11.2 API device and immediate context.
device.As(&m_d3dDevice);

context.As(&m_d3dContext);

```

Create a swap chain

Direct3D 11 includes a device API called DirectX graphics infrastructure (DXGI). The DXGI interface allows us to (for example) control how the swap chain is configured and set up shared devices. At this step in initializing Direct3D, we're going to use DXGI to create a swap chain. Since we created the device, we can follow an interface chain back to the DXGI adapter.

The Direct3D device implements a COM interface for DXGI. First we need to get that interface and use it to request the DXGI adapter hosting the device. Then we use the DXGI adapter to create a DXGI factory.

Note These are COM interfaces so your first response might be to use [QueryInterface](#). You should use [Microsoft::WRL::ComPtr](#) smart pointers instead. Then just call the [As\(\)](#) method, supplying an empty COM pointer of the correct interface type.

C++

```
ComPtr<IDXGIDevice2> dxgiDevice;  
m_d3dDevice.As(&dxgiDevice);  
  
// Then, the adapter hosting the device;  
ComPtr<IDXGIAdapter> dxgiAdapter;  
dxgiDevice->GetAdapter(&dxgiAdapter);  
  
// Then, the factory that created the adapter interface:  
ComPtr<IDXGIFactory2> dxgiFactory;  
dxgiAdapter->GetParent(  
    __uuidof(IDXGIFactory2),  
    &dxgiFactory  
);
```

Now that we have the DXGI factory, we can use it to create the swap chain. Let's define the swap chain parameters. We need to specify the surface format; we'll choose [DXGI_FORMAT_B8G8R8A8_UNORM](#) because it's compatible with Direct2D. We'll turn off display scaling, multisampling, and stereo rendering because they aren't used in this example. Since we are running directly in a CoreWindow we can leave the width and height set to 0 and get full-screen values automatically.

Note Always set the *SDKVersion* parameter to `D3D11_SDK_VERSION` for UWP apps.

Direct3D 11

C++

```
ComPtr<IDXGISwapChain1> swapChain;  
dxgiFactory->CreateSwapChainForCoreWindow(  
    m_d3dDevice.Get(),  
    reinterpret_cast<IUnknown*>(window),  
    &swapChainDesc,  
    nullptr,  
    &swapChain  
);  
swapChain.As(&m_swapChain);
```

To ensure we aren't rendering more often than the screen can actually display, we set frame latency to 1 and use [DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL](#). This saves power and is a store certification requirement; we'll learn more about presenting to the screen in part 2 of this walkthrough.

Note You can use multithreading (for example, [ThreadPool](#) work items) to continue other work while the rendering thread is blocked.

Direct3D 11

C++

```
dxgiDevice->SetMaximumFrameLatency(1);
```

Now we can set up the back buffer for rendering.

Configure the back buffer as a render target

First we have to get a handle to the back buffer. (Note that the back buffer is owned by the DXGI swap chain, whereas in DirectX 9 it was owned by the Direct3D device.) Then we tell the Direct3D device to use it as the render target by creating a render target *view* using the back buffer.

Direct3D 11

C++

```
ComPtr<ID3D11Texture2D> backBuffer;  
m_swapChain->GetBuffer(  
    0,  
    __uuidof(ID3D11Texture2D),  
    &backBuffer  
);  
  
// Create a render target view on the back buffer.  
m_d3dDevice->CreateRenderTargetView(  
    backBuffer.Get(),  
    nullptr,  
    &m_renderTargetView  
);
```

Now the device context comes into play. We tell Direct3D to use our newly-created render target view by using the device context interface. We'll retrieve the width and height of the back buffer so that we can target the whole window as our viewport. Note that the back buffer is attached to the swap chain, so if the window size changes (for example, the user drags the game window to another monitor) the back buffer will need to be resized and some setup will need to be redone.

Direct3D 11

C++

```
D3D11_TEXTURE2D_DESC backBufferDesc = {0};
backBuffer->GetDesc(&backBufferDesc);

CD3D11_VIEWPORT viewport(
    0.0f,
    0.0f,
    static_cast<float>(backBufferDesc.Width),
    static_cast<float>(backBufferDesc.Height)
);

m_d3dContext->RSSetViewports(1, &viewport);
```

Now that we have a device handle and a full-screen render target, we are ready to load and draw geometry. Continue to [Part 2: Rendering](#).

Convert the rendering framework

Article • 10/20/2022

Summary

- [Part 1: Initialize Direct3D 11](#)
- Part 2: Convert the rendering framework
- [Part 3: Port the game loop](#)

Shows how to convert a simple rendering framework from Direct3D 9 to Direct3D 11, including how to port geometry buffers, how to compile and load HLSL shader programs, and how to implement the rendering chain in Direct3D 11. Part 2 of the [Port a simple Direct3D 9 app to DirectX 11 and Universal Windows Platform \(UWP\)](#) walkthrough.

Convert effects to HLSL shaders

The following example is a simple D3DX technique, written for the legacy Effects API, for hardware vertex transformation and pass-through color data.

Direct3D 9 shader code

C++

```
// Global variables
matrix g_mWorld;           // world matrix for object
matrix g_View;             // view matrix
matrix g_Projection;       // projection matrix

// Shader pipeline structures
struct VS_OUTPUT
{
    float4 Position        : POSITION;    // vertex position
    float4 Color            : COLOR0;    // vertex diffuse color
};

struct PS_OUTPUT
{
    float4 RGBColor : COLOR0; // Pixel color
};

// Vertex shader
VS_OUTPUT RenderSceneVS(float3 vPos : POSITION,
                        float3 vColor : COLOR0)
{
    VS_OUTPUT Output;
```

```

float4 pos = float4(vPos, 1.0f);

// Transform the position from object space to homogeneous projection
space
pos = mul(pos, g_mWorld);
pos = mul(pos, g_View);
pos = mul(pos, g_Projection);

Output.Position = pos;

// Just pass through the color data
Output.Color = float4(vColor, 1.0f);

return Output;
}

// Pixel shader
PS_OUTPUT RenderScenePS(VS_OUTPUT In)
{
    PS_OUTPUT Output;

    Output.RGBColor = In.Color;

    return Output;
}

// Technique
technique RenderSceneSimple
{
    pass P0
    {
        VertexShader = compile vs_2_0 RenderSceneVS();
        PixelShader   = compile ps_2_0 RenderScenePS();
    }
}

```

In Direct3D 11, we can still use our HLSL shaders. We put each shader in its own HLSL file so that Visual Studio compiles them into separate files and later, we'll load them as separate Direct3D resources. We set the target level to [Shader Model 4 Level 9_1 \(/4_0_level_9_1\)](#) because these shaders are written for DirectX 9.1 GPUs.

When we defined the input layout, we made sure it represented the same data structure we use to store per-vertex data in system memory and in GPU memory. Similarly, the output of a vertex shader should match the structure used as input to the pixel shader. The rules are not the same as passing data from one function to another in C++; you can omit unused variables at the end of the structure. But the order can't be rearranged and you can't skip content in the middle of the data structure.

Note The rules in Direct3D 9 for binding vertex shaders to pixel shaders were more relaxed than the rules in Direct3D 11. The Direct3D 9 arrangement was flexible, but

inefficient.

It's possible that your HLSL files uses older syntax for shader semantics - for example, COLOR instead of SV_TARGET. If so you'll need to enable HLSL compatibility mode (/Gec compiler option) or update the shader [semantics](#) to the current syntax. The vertex shader in this example has been updated with current syntax.

Here's our hardware transformation vertex shader, this time defined in its own file.

Note Vertex shaders are required to output the SV_POSITION system value semantic. This semantic resolves the vertex position data to coordinate values where x is between -1 and 1, y is between -1 and 1, z is divided by the original homogeneous coordinate w value (z/w), and w is 1 divided by the original w value (1/w).

HLSL vertex shader (feature level 9.1)

C++

```
cbuffer ModelViewProjectionConstantBuffer : register(b0)
{
    matrix mWorld;          // world matrix for object
    matrix View;            // view matrix
    matrix Projection;      // projection matrix
};

struct VS_INPUT
{
    float3 vPos    : POSITION;
    float3 vColor  : COLOR0;
};

struct VS_OUTPUT
{
    float4 Position : SV_POSITION; // Vertex shaders must output SV_POSITION
    float4 Color    : COLOR0;
};

VS_OUTPUT main(VS_INPUT input) // main is the default function name
{
    VS_OUTPUT Output;

    float4 pos = float4(input.vPos, 1.0f);

    // Transform the position from object space to homogeneous projection
```

```

space
    pos = mul(pos, mWorld);
    pos = mul(pos, View);
    pos = mul(pos, Projection);
    Output.Position = pos;

    // Just pass through the color data
    Output.Color = float4(input.vColor, 1.0f);

    return Output;
}

```

This is all we need for our pass-through pixel shader. Even though we call it a pass-through, it's actually getting perspective-correct interpolated color data for each pixel. Note that the SV_TARGET system value semantic is applied to the color value output by our pixel shader as required by the API.

Note Shader level 9_x pixel shaders cannot read from the SV_POSITION system value semantic. Model 4.0 (and higher) pixel shaders can use SV_POSITION to retrieve the pixel location on the screen, where x is between 0 and the render target width and y is between 0 and the render target height (each offset by 0.5).

Most pixel shaders are much more complex than a pass through; note that higher Direct3D feature levels allow a much greater number of calculations per shader program.

HLSL pixel shader (feature level 9.1)

```

C++

struct PS_INPUT
{
    float4 Position : SV_POSITION; // interpolated vertex position (system
    value)
    float4 Color : COLOR0; // interpolated diffuse color
};

struct PS_OUTPUT
{
    float4 RGBColor : SV_TARGET; // pixel color (your PS computes this
    system value)
};

PS_OUTPUT main(PS_INPUT In)
{
    PS_OUTPUT Output;

```



```
Output.RGBColor = In.Color;  
  
return Output;  
}
```

Compile and load shaders

Direct3D 9 games often used the Effects library as a convenient way to implement programmable pipelines. Effects could be compiled at run-time using the [D3DXCreateEffectFromFile function](#) method.

Loading an effect in Direct3D 9

C++

```
// Turn off preshader optimization to keep calculations on the GPU  
DWORD dwShaderFlags = D3DXSHADER_NO_PRESHADER;  
  
// Only enable debug info when compiling for a debug target  
#if defined (DEBUG) || defined (_DEBUG)  
dwShaderFlags |= D3DXSHADER_DEBUG;  
#endif  
  
D3DXCreateEffectFromFile(  
    m_pd3dDevice,  
    L"CubeShaders.fx",  
    NULL,  
    NULL,  
    dwShaderFlags,  
    NULL,  
    &m_pEffect,  
    NULL  
);
```

Direct3D 11 works with shader programs as binary resources. Shaders are compiled when the project is built and then treated as resources. So our example will load the shader bytecode into system memory, use the Direct3D device interface to create a Direct3D resource for each shader, and point to the Direct3D shader resources when we set up each frame.

Loading a shader resource in Direct3D 11

C++

```
// BasicReaderWriter is a tested file loader used in SDK samples.  
BasicReaderWriter^ readerWriter = ref new BasicReaderWriter();
```

```

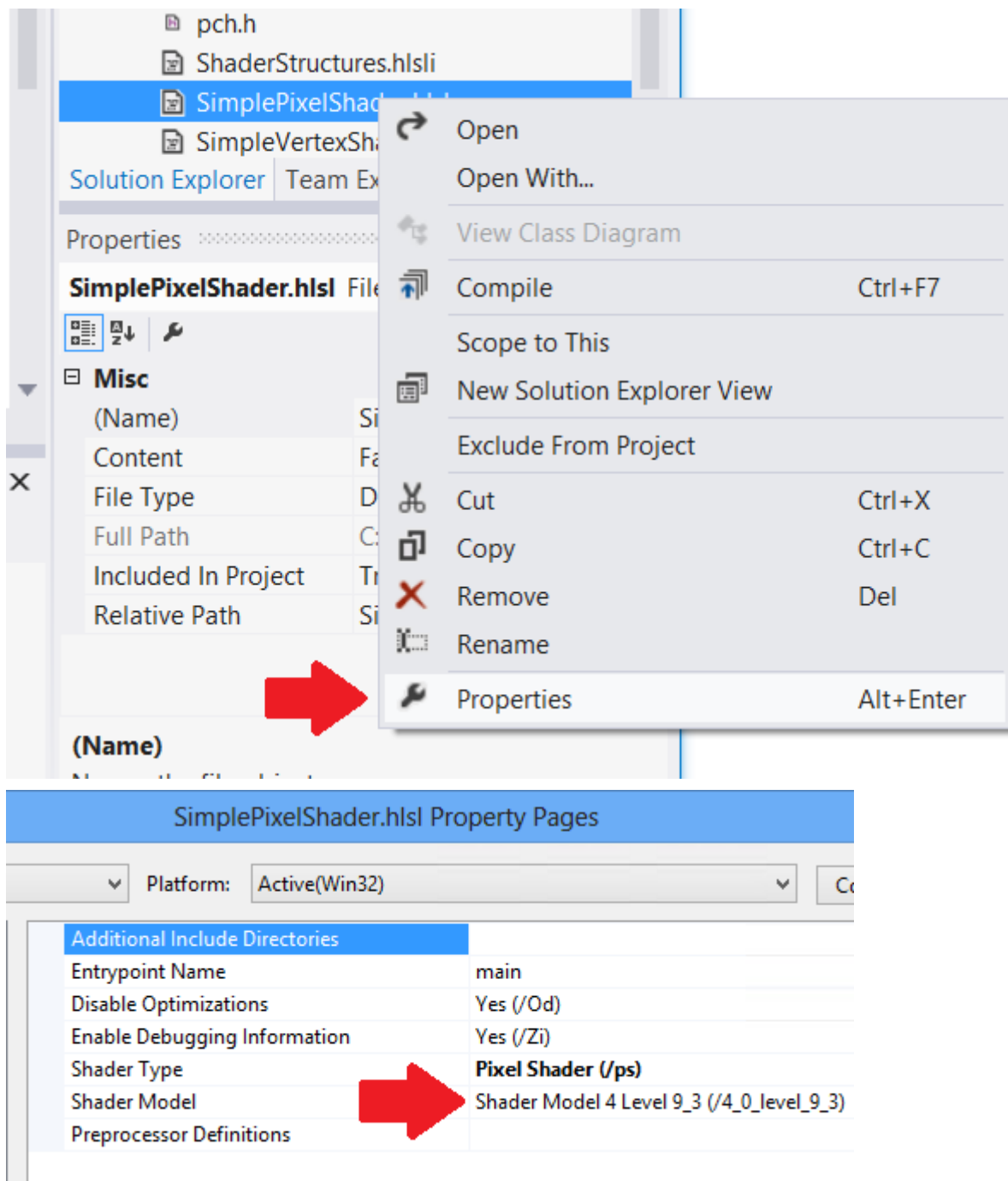
// Load vertex shader:
Platform::Array<byte>^ vertexShaderData =
    readerWriter->ReadData("CubeVertexShader.cso");

// This call allocates a device resource, validates the vertex shader
// with the device feature level, and stores the vertex shader bits in
// graphics memory.
m_d3dDevice->CreateVertexShader(
    vertexShaderData->Data,
    vertexShaderData->Length,
    nullptr,
    &m_vertexShader
);

```

To include the shader bytecode in your compiled app package, just add the HLSL file to the Visual Studio project. Visual Studio will use the [Effect-Compiler Tool](#) (FXC) to compile HLSL files into compiled shader objects (.CSO files) and include them in the app package.

Note Be sure to set the correct target feature level for the HLSL compiler: right-click the HLSL source file in Visual Studio, select Properties, and change the **Shader Model** setting under **HLSL Compiler -> General**. Direct3D checks this property against the hardware capabilities when your app creates the Direct3D shader resource.



This is a good place to create the input layout, which corresponds to the vertex stream declaration in Direct3D 9. The per-vertex data structure needs to match what the vertex shader uses; in Direct3D 11 we have more control over the input layout; we can define the array size and bit length of floating-point vectors and specify semantics for the vertex shader. We create a [D3D11_INPUT_ELEMENT_DESC](#) structure and use it to inform Direct3D what the per-vertex data will look like. We waited until after we loaded the vertex shader to define the input layout because the API validates the input layout against the vertex shader resource. If the input layout isn't compatible then Direct3D throws an exception.

Per-vertex data has to be stored in compatible types in system memory. DirectXMath data types can help; for example, `DXGI_FORMAT_R32G32B32_FLOAT` corresponds to [XMFLOAT3](#).

Note Constant buffers use a fixed input layout that aligns to four floating-point numbers at a time. [XMFLOAT4](#) (and its derivatives) are recommended for constant buffer data.

Setting the input layout in Direct3D 11

C++

```
// Create input layout:
const D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT,
      0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },

    { "COLOR", 0, DXGI_FORMAT_R32G32B32_FLOAT,
      0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0 },
};
```

Create geometry resources

In Direct3D 9 we stored geometry resources by creating buffers on the Direct3D device, locking the memory, and copying data from CPU memory to GPU memory.

Direct3D 9

C++

```
// Create vertex buffer:
VOID* pVertices;

// In Direct3D 9 we create the buffer, lock it, and copy the data from
// system memory to graphics memory.
m_pd3dDevice->CreateVertexBuffer(
    sizeof(CubeVertices),
    0,
    D3DFVF_XYZ | D3DFVF_DIFFUSE,
    D3DPPOOL_MANAGED,
    &pVertexBuffer,
    NULL);

pVertexBuffer->Lock(
    0,
    sizeof(CubeVertices),
    &pVertices,
    0);
```

```
memcpy(pVertices, CubeVertices, sizeof(CubeVertices));  
pVertexBuffer->Unlock();
```

DirectX 11 follows a simpler process. The API automatically copies the data from system memory to the GPU. We can use COM smart pointers to help make programming easier.

DirectX 11

C++

```
D3D11_SUBRESOURCE_DATA vertexBufferData = {0};  
vertexBufferData.pSysMem = CubeVertices;  
vertexBufferData.SysMemPitch = 0;  
vertexBufferData.SysMemSlicePitch = 0;  
CD3D11_BUFFER_DESC vertexBufferDesc(  
    sizeof(CubeVertices),  
    D3D11_BIND_VERTEX_BUFFER);  
  
// This call allocates a device resource for the vertex buffer and copies  
// in the data.  
m_d3dDevice->CreateBuffer(  
    &vertexBufferDesc,  
    &vertexBufferData,  
    &m_vertexBuffer  
);
```

Implement the rendering chain

Direct3D 9 games often used an effect-based rendering chain. This type of rendering chain sets up the effect object, provides it with the resources it needs, and lets it render each pass.

Direct3D 9 rendering chain

C++

```
// Clear the render target and the z-buffer.  
m_pd3dDevice->Clear(  
    0, NULL,  
    D3DCLEAR_TARGET | D3DCLEAR_ZBUFFER,  
    D3DCOLOR_ARGB(0, 45, 50, 170),  
    1.0f, 0  
);  
  
// Set the effect technique  
m_pEffect->SetTechnique("RenderSceneSimple");  
  
// Rotate the cube 1 degree per frame.  
D3DXMATRIX world;
```

```

D3DXMatrixRotationY(&world, D3DXToRadian(m_frameCount++));

// Set the matrices up using traditional functions.
m_pEffect->SetMatrix("g_mWorld", &world);
m_pEffect->SetMatrix("g_View", &m_view);
m_pEffect->SetMatrix("g_Projection", &m_projection);

// Render the scene using the Effects library.
if(SUCCEEDED(m_pd3dDevice->BeginScene()))
{
    // Begin rendering effect passes.
    UINT passes = 0;
    m_pEffect->Begin(&passes, 0);

    for (UINT i = 0; i < passes; i++)
    {
        m_pEffect->BeginPass(i);

        // Send vertex data to the pipeline.
        m_pd3dDevice->SetFVF(D3DFVF_XYZ | D3DFVF_DIFFUSE);
        m_pd3dDevice->SetStreamSource(
            0, pVertexBuffer,
            0, sizeof(VertexPositionColor)
        );
        m_pd3dDevice->SetIndices(pIndexBuffer);

        // Draw the cube.
        m_pd3dDevice->DrawIndexedPrimitive(
            D3DPT_TRIANGLELIST,
            0, 0, 8, 0, 12
        );
        m_pEffect->EndPass();
    }
    m_pEffect->End();

    // End drawing.
    m_pd3dDevice->EndScene();
}

// Present frame:
// Show the frame on the primary surface.
m_pd3dDevice->Present(NULL, NULL, NULL, NULL);

```

The DirectX 11 rendering chain will still do the same tasks, but the rendering passes need to be implemented differently. Instead of putting the specifics in FX files and letting the rendering techniques be more-or-less opaque to our C++ code, we'll set up all our rendering in C++.

Here's how our rendering chain will look. We need to supply the input layout we created after loading the vertex shader, supply each of the shader objects, and specify the constant buffers for each shader to use. This example doesn't include multiple rendering

passes, but if it did we'd do a similar rendering chain for each pass, changing the setup as needed.

Direct3D 11 rendering chain

C++

```
// Clear the back buffer.
const float midnightBlue[] = { 0.098f, 0.098f, 0.439f, 1.000f };
m_d3dContext->ClearRenderTargetView(
    m_renderTargetView.Get(),
    midnightBlue
);

// Set the render target. This starts the drawing operation.
m_d3dContext->OMSetRenderTargets(
    1, // number of render target views for this drawing operation.
    m_renderTargetView.GetAddressOf(),
    nullptr
);

// Rotate the cube 1 degree per frame.
XMStoreFloat4x4(
    &m_constantBufferData.model,
    XMMatrixTranspose(XMMatrixRotationY(m_frameCount++ * XM_PI / 180.f))
);

// Copy the updated constant buffer from system memory to video memory.
m_d3dContext->UpdateSubresource(
    m_constantBuffer.Get(),
    0, // update the 0th subresource
    NULL, // use the whole destination
    &m_constantBufferData,
    0, // default pitch
    0 // default pitch
);

// Send vertex data to the Input Assembler stage.
UINT stride = sizeof(VertexPositionColor);
UINT offset = 0;

m_d3dContext->IASetVertexBuffers(
    0, // start with the first vertex buffer
    1, // one vertex buffer
    m_vertexBuffer.GetAddressOf(),
    &stride,
    &offset
);

m_d3dContext->IASetIndexBuffer(
    m_indexBuffer.Get(),
```

```

    DXGI_FORMAT_R16_UINT,
    0    // no offset
);

m_d3dContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
m_d3dContext->IASetInputLayout(m_inputLayout.Get());

// Set the vertex shader.
m_d3dContext->VSSetShader(
    m_vertexShader.Get(),
    nullptr,
    0
);

// Set the vertex shader constant buffer data.
m_d3dContext->VSSetConstantBuffers(
    0,    // register 0
    1,    // one constant buffer
    m_constantBuffer.GetAddressOf()
);

// Set the pixel shader.
m_d3dContext->PSSetShader(
    m_pixelShader.Get(),
    nullptr,
    0
);

// Draw the cube.
m_d3dContext->DrawIndexed(
    m_indexCount,
    0,    // start with index 0
    0    // start with vertex 0
);

```

The swap chain is part of graphics infrastructure, so we use our DXGI swap chain to present the completed frame. DXGI blocks the call until the next vsync; then it returns, and our game loop can continue to the next iteration.

Presenting a frame to the screen using DirectX 11

C++

```
m_swapChain->Present(1, 0);
```

The rendering chain we just created will be called from a game loop implemented in the [IFrameworkView::Run](#) method. This is shown in [Part 3: Viewport and game loop](#).

Port the game loop

Article • 10/20/2022

Summary

- [Part 1: Initialize Direct3D 11](#)
- [Part 2: Convert the rendering framework](#)
- Part 3: Port the game loop

Shows how to implement a window for a Universal Windows Platform (UWP) game and how to bring over the game loop, including how to build an [IFrameworkView](#) to control a full-screen [CoreWindow](#). Part 3 of the [Port a simple Direct3D 9 app to DirectX 11 and UWP](#) walkthrough.

Create a window

To set up a desktop window with a Direct3D 9 viewport, we had to implement the traditional windowing framework for desktop apps. We had to create an HWND, set the window size, provide a window processing callback, make it visible, and so on.

The UWP environment has a much simpler system. Instead of setting up a traditional window, a Microsoft Store game using DirectX implements [IFrameworkView](#). This interface exists for DirectX apps and games to run directly in a [CoreWindow](#) inside the app container.

Note Windows supplies managed pointers to resources such as the source application object and the [CoreWindow](#). See [Handle to Object Operator \(^\)](#).

Your "main" class needs to inherit from [IFrameworkView](#) and implement the five [IFrameworkView](#) methods: [Initialize](#), [SetWindow](#), [Load](#), [Run](#), and [Uninitialize](#). In addition to creating the [IFrameworkView](#), which is (essentially) where your game will reside, you need to implement a factory class that creates an instance of your [IFrameworkView](#). Your game still has an executable with a method called `main()`, but all `main` can do is use the factory to create the [IFrameworkView](#) instance.

Main function

C++

```
//-----
---
// Required method for a DirectX-only app.
// The main function is only used to initialize the app's IFrameworkView
// class.
//-----
---
[Platform::MTAThread]
int main(Platform::Array<Platform::String^>^)
{
    auto direct3DApplicationSource = ref new Direct3DApplicationSource();
    CoreApplication::Run(direct3DApplicationSource);
    return 0;
}
```

IFrameworkView factory

```
C++

//-----
---
// This class creates our IFrameworkView.
//-----
---
ref class Direct3DApplicationSource sealed :
    Windows::ApplicationModel::Core::IFrameworkViewSource
{
public:
    virtual Windows::ApplicationModel::Core::IFrameworkView^ CreateView()
    {
        return ref new Cube11();
    };
};
```

Port the game loop

Let's look at the game loop from our Direct3D 9 implementation. This code exists in the app's main function. Each iteration of this loop processes a window message or renders a frame.

Game loop in Direct3D 9 desktop game

```
C++

while(WM_QUIT != msg.message)
{
    // Process window events.
    // Use PeekMessage() so we can use idle time to render the scene.
```

```

bGotMsg = (PeekMessage(&msg, NULL, 0U, 0U, PM_REMOVE) != 0);

if(bGotMsg)
{
    // Translate and dispatch the message
    TranslateMessage(&msg);
    DispatchMessage(&msg);
}
else
{
    // Render a new frame.
    // Render frames during idle time (when no messages are waiting).
    RenderFrame();
}
}

```

The game loop is similar - but easier - in the UWP version of our game:

The game loop goes in the `IFrameworkView::Run` method (instead of `main()`) because our game functions within the `IFrameworkView` class.

Instead of implementing a message handling framework and calling `PeekMessage`, we can call the `ProcessEvents` method built in to our app window's `CoreDispatcher`. There's no need for the game loop to branch and handle messages - just call `ProcessEvents` and proceed.

Game loop in Direct3D 11 Microsoft Store game

```

C++

// UWP apps should not exit. Use app lifecycle events instead.
while (true)
{
    // Process window events.
    auto dispatcher = CoreWindow::GetForCurrentThread()->Dispatcher;
    dispatcher->ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

    // Render a new frame.
    RenderFrame();
}

```

Now we have a UWP app that sets up the same basic graphics infrastructure, and renders the same colorful cube, as our DirectX 9 example.

Where do I go from here?

Bookmark the [DirectX 11 porting FAQ](#).

The DirectX UWP templates include a robust Direct3D device infrastructure that's ready for use with your game. See [Create a DirectX game project from a template](#) for guidance on picking the right template.

Visit the following in-depth Microsoft Store game development articles:

- [Walkthrough: a simple UWP game with DirectX](#)
- [Audio for games](#)
- [Move-look controls for games](#)

Walkthrough: Implement shadow volumes using depth buffers in Direct3D 11

Article • 10/20/2022

This walkthrough demonstrates how to render shadow volumes using depth maps, using Direct3D 11 on devices of all Direct3D feature levels.

Topic	Description
Create depth buffer device resources	Learn how to create the Direct3D device resources necessary to support depth testing for shadow volumes.
Render the shadow map to the depth buffer	Render from the point of view of the light to create a two-dimensional depth map representing the shadow volume.
Render the scene with depth testing	Create a shadow effect by adding depth testing to your vertex (or geometry) shader and your pixel shader.
Support shadow maps on a range of hardware	Render higher-fidelity shadows on faster devices and faster shadows on less powerful devices.

Shadow mapping application to Direct3D 9 desktop porting

Windows 8 added depth comparison functionality to feature level 9_1 and 9_3. Now you can migrate rendering code with shadow volumes to DirectX 11, and the Direct3D 11 renderer will be downlevel compatible with feature level 9 devices. This walkthrough shows how any Direct3D 11 app or game can implement traditional shadow volumes using depth testing. The code covers the following process:

1. Creating Direct3D device resources for shadow mapping.
2. Adding a rendering pass to create the depth map.
3. Adding depth testing to the main rendering pass.
4. Implementing the necessary shader code.

5. Options for fast rendering on downlevel hardware.

Upon completing this walkthrough, you should be familiar with how to implement a basic compatible shadow volume technique in Direct3D 11 that's compatible with feature level 9_1 and above.

Prerequisites

You should [Prepare your dev environment for Universal Windows Platform \(UWP\) DirectX game development](#). You don't need a template yet, but you'll need Microsoft Visual Studio 2015 to build the code sample for this walkthrough.

Related topics

Direct3D

- [Writing HLSL Shaders in Direct3D 9](#)
- [Create a new DirectX 11 project for UWP](#)

Shadow mapping technical articles

- [Common Techniques to Improve Shadow Depth Maps](#)
- [Cascaded Shadow Maps](#)

Create depth buffer device resources

Article • 10/20/2022

Learn how to create the Direct3D device resources necessary to support depth testing for shadow volumes. Part 1 of [Walkthrough: Implement shadow volumes using depth buffers in Direct3D 11](#).

Resources you'll need

Rendering a depth map for shadow volumes requires the following Direct3D device-dependent resources:

- A resource (buffer) for the depth map
- A depth stencil view and shader resource view for the resource
- A comparison sampler state object
- Constant buffers for light POV matrices
- A viewport for rendering the shadow map (typically a square viewport)
- A rendering state object to enable front face culling
- You will also need a rendering state object to switch back to back face culling, if you don't already use one.

Note that creation of these resources needs to be included in a device-dependent resource creation routine, that way your renderer can recreate them if (for example) a new device driver is installed, or the user moves your app to a monitor attached to a different graphics adapter.

Check feature support

Before creating the depth map, call the [CheckFeatureSupport](#) method on the Direct3D device, request `D3D11_FEATURE_D3D9_SHADOW_SUPPORT`, and provide a [D3D11_FEATURE_DATA_D3D9_SHADOW_SUPPORT](#) structure.

C++

```
D3D11_FEATURE_DATA_D3D9_SHADOW_SUPPORT isD3D9ShadowSupported;  
ZeroMemory(&isD3D9ShadowSupported, sizeof(isD3D9ShadowSupported));  
pD3DDevice->CheckFeatureSupport(  
    D3D11_FEATURE_D3D9_SHADOW_SUPPORT,  
    &isD3D9ShadowSupported,  
    sizeof(D3D11_FEATURE_D3D9_SHADOW_SUPPORT)  
);
```



```

if
(isD3D9ShadowSupported.SupportsDepthAsTextureWithLessEqualComparisonFilter)
{
    // Init shadow map resources

```

If this feature is not supported, do not try to load shaders compiled for shader model 4 level 9_x that call sample comparison functions. In many cases, lack of support for this feature means that the GPU is a legacy device with a driver that isn't updated to support at least WDDM 1.2. If the device supports at least feature level 10_0 then you can load a sample comparison shader compiled for shader model 4_0 instead.

Create depth buffer

First, try creating the depth map with a higher-precision depth format. Set up matching shader resource view properties first. If the resource creation fails, for example due to low device memory or a format that the hardware doesn't support, try a lower-precision format and change properties to match.

This step is optional if you only need a low-precision depth format, for example when rendering on medium-resolution Direct3D feature level 9_1 devices.

C++

```

D3D11_TEXTURE2D_DESC shadowMapDesc;
ZeroMemory(&shadowMapDesc, sizeof(D3D11_TEXTURE2D_DESC));
shadowMapDesc.Format = DXGI_FORMAT_R24G8_TYPELESS;
shadowMapDesc.MipLevels = 1;
shadowMapDesc.ArraySize = 1;
shadowMapDesc.SampleDesc.Count = 1;
shadowMapDesc.BindFlags = D3D11_BIND_SHADER_RESOURCE |
D3D11_BIND_DEPTH_STENCIL;
shadowMapDesc.Height = static_cast<UINT>(m_shadowMapDimension);
shadowMapDesc.Width = static_cast<UINT>(m_shadowMapDimension);

HRESULT hr = pD3DDevice->CreateTexture2D(
    &shadowMapDesc,
    nullptr,
    &m_shadowMap
);

```

Then create the resource views. Set the mip slice to zero on the depth stencil view and set mip levels to 1 on the shader resource view. Both have a texture dimension of TEXTURE2D, and both need to use a matching [DXGI_FORMAT](#).

C++

```

D3D11_DEPTH_STENCIL_VIEW_DESC depthStencilViewDesc;
ZeroMemory(&depthStencilViewDesc, sizeof(D3D11_DEPTH_STENCIL_VIEW_DESC));
depthStencilViewDesc.Format = DXGI_FORMAT_D24_UNORM_S8_UINT;
depthStencilViewDesc.ViewDimension = D3D11_DSV_DIMENSION_TEXTURE2D;
depthStencilViewDesc.Texture2D.MipSlice = 0;

D3D11_SHADER_RESOURCE_VIEW_DESC shaderResourceViewDesc;
ZeroMemory(&shaderResourceViewDesc,
sizeof(D3D11_SHADER_RESOURCE_VIEW_DESC));
shaderResourceViewDesc.ViewDimension = D3D11_SRV_DIMENSION_TEXTURE2D;
shaderResourceViewDesc.Format = DXGI_FORMAT_R24_UNORM_X8_TYPELESS;
shaderResourceViewDesc.Texture2D.MipLevels = 1;

hr = pD3DDevice->CreateDepthStencilView(
    m_shadowMap.Get(),
    &depthStencilViewDesc,
    &m_shadowDepthView
);

hr = pD3DDevice->CreateShaderResourceView(
    m_shadowMap.Get(),
    &shaderResourceViewDesc,
    &m_shadowResourceView
);

```

Create comparison state

Now create the comparison sampler state object. Feature level 9_1 only supports D3D11_COMPARISON_LESS_EQUAL. Filtering choices are explained more in [Supporting shadow maps on a range of hardware](#) - or you can just pick point filtering for faster shadow maps.

Note that you can specify the D3D11_TEXTURE_ADDRESS_BORDER address mode and it will work on feature level 9_1 devices. This applies to pixel shaders that don't test whether the pixel is in the light's view frustum before doing the depth test. By specifying 0 or 1 for each border, you can control whether pixels outside the light's view frustum pass or fail the depth test, and therefore whether they are lit or in shadow.

On feature level 9_1, the following required values must be set: **MinLOD** is set to zero, **MaxLOD** is set to **D3D11_FLOAT32_MAX**, and **MaxAnisotropy** is set to zero.

C++

```

D3D11_SAMPLER_DESC comparisonSamplerDesc;
ZeroMemory(&comparisonSamplerDesc, sizeof(D3D11_SAMPLER_DESC));
comparisonSamplerDesc.AddressU = D3D11_TEXTURE_ADDRESS_BORDER;
comparisonSamplerDesc.AddressV = D3D11_TEXTURE_ADDRESS_BORDER;
comparisonSamplerDesc.AddressW = D3D11_TEXTURE_ADDRESS_BORDER;

```

```

comparisonSamplerDesc.BorderColor[0] = 1.0f;
comparisonSamplerDesc.BorderColor[1] = 1.0f;
comparisonSamplerDesc.BorderColor[2] = 1.0f;
comparisonSamplerDesc.BorderColor[3] = 1.0f;
comparisonSamplerDesc.MinLOD = 0.f;
comparisonSamplerDesc.MaxLOD = D3D11_FLOAT32_MAX;
comparisonSamplerDesc.MipLODBias = 0.f;
comparisonSamplerDesc.MaxAnisotropy = 0;
comparisonSamplerDesc.ComparisonFunc = D3D11_COMPARISON_LESS_EQUAL;
comparisonSamplerDesc.Filter = D3D11_FILTER_COMPARISON_MIN_MAG_MIP_POINT;

// Point filtered shadows can be faster, and may be a good choice when
// rendering on hardware with lower feature levels. This sample has a
// UI option to enable/disable filtering so you can see the difference
// in quality and speed.

DX::ThrowIfFailed(
    pD3DDevice->CreateSamplerState(
        &comparisonSamplerDesc,
        &m_comparisonSampler_point
    )
);

```

Create render states

Now create a render state you can use to enable front face culling. Note that feature level 9_1 devices require **DepthClipEnable** set to **true**.

```

C++

D3D11_RASTERIZER_DESC drawingRenderStateDesc;
ZeroMemory(&drawingRenderStateDesc, sizeof(D3D11_RASTERIZER_DESC));
drawingRenderStateDesc.CullMode = D3D11_CULL_BACK;
drawingRenderStateDesc.FillMode = D3D11_FILL_SOLID;
drawingRenderStateDesc.DepthClipEnable = true; // Feature level 9_1 requires
DepthClipEnable == true
DX::ThrowIfFailed(
    pD3DDevice->CreateRasterizerState(
        &drawingRenderStateDesc,
        &m_drawingRenderState
    )
);

```

Create a render state you can use to enable back face culling. If your rendering code already turns on back face culling, then you can skip this step.

```

C++

```

```

D3D11_RASTERIZER_DESC shadowRenderStateDesc;
ZeroMemory(&shadowRenderStateDesc, sizeof(D3D11_RASTERIZER_DESC));
shadowRenderStateDesc.CullMode = D3D11_CULL_FRONT;
shadowRenderStateDesc.FillMode = D3D11_FILL_SOLID;
shadowRenderStateDesc.DepthClipEnable = true;

DX::ThrowIfFailed(
    pD3DDevice->CreateRasterizerState(
        &shadowRenderStateDesc,
        &m_shadowRenderState
    )
);

```

Create constant buffers

Don't forget to create a constant buffer for rendering from the light's point of view. You can also use this constant buffer to specify the light position to the shader. Use a perspective matrix for point lights, and use an orthogonal matrix for directional lights (such as sunlight).

C++

```

DX::ThrowIfFailed(
    m_deviceResources->GetD3DDevice()->CreateBuffer(
        &viewProjectionConstantBufferDesc,
        nullptr,
        &m_lightViewProjectionBuffer
    )
);

```

Fill the constant buffer data. Update the constant buffers once during initialization, and again if the light values have changed since the previous frame.

C++

```

{
    XMATRIX lightPerspectiveMatrix = XMMatrixPerspectiveFovRH(
        XM_PIDIV2,
        1.0f,
        12.f,
        50.f
    );

    XMStoreFloat4x4(
        &m_lightViewProjectionBufferData.projection,
        XMMatrixTranspose(lightPerspectiveMatrix)
    );
}

```

```

    // Point light at (20, 15, 20), pointed at the origin. POV up-vector is
    // along the y-axis.
    static const XMVECTORF32 eye = { 20.0f, 15.0f, 20.0f, 0.0f };
    static const XMVECTORF32 at = { 0.0f, 0.0f, 0.0f, 0.0f };
    static const XMVECTORF32 up = { 0.0f, 1.0f, 0.0f, 0.0f };

    XMStoreFloat4x4(
        &m_lightViewProjectionBufferData.view,
        XMMatrixTranspose(XMMatrixLookAtRH(eye, at, up))
    );

    // Store the light position to help calculate the shadow offset.
    XMStoreFloat4(&m_lightViewProjectionBufferData.pos, eye);
}

```

C++

```

context->UpdateSubresource(
    m_lightViewProjectionBuffer.Get(),
    0,
    NULL,
    &m_lightViewProjectionBufferData,
    0,
    0
);

```

Create a viewport

You need a separate viewport to render to the shadow map. The viewport isn't a device-based resource; you're free to create it elsewhere in your code. Creating the viewport along with the shadow map can help make it more convenient to keep the dimension of the viewport congruent with the shadow map dimension.

C++

```

// Init viewport for shadow rendering
ZeroMemory(&m_shadowViewport, sizeof(D3D11_VIEWPORT));
m_shadowViewport.Height = m_shadowMapDimension;
m_shadowViewport.Width = m_shadowMapDimension;
m_shadowViewport.MinDepth = 0.f;
m_shadowViewport.MaxDepth = 1.f;

```

In the next part of this walkthrough, learn how to create the shadow map by [rendering to the depth buffer](#).